

Week 4 — Lab Guide (2-Day)

LLM Application Foundations + Model Landscape

Phase 1 • Foundations, Eval Literacy & System Design

What you'll have built by Friday

The `/ask` and `/ask_batched` endpoints from Week 3 are unchanged in their contract — same URLs, same `Question` in, same `Answer` out. By the end of this lab their *insides* are different:

- `ask_llm` uses **tool-calling** to get structured outputs from the model. Your `Answer` Pydantic model gains three new fields — `confidence`, `sources`, and `schema_version` — without breaking any W3 client.
- `/ask` streams **real OpenAI chunks** instead of the W3 word-by-word `asyncio.sleep` simulation.
- Every `/ask_batched` call writes a row to SQLite with the **real cost** (from `response.usage`) and the **model name** that produced it.
- A 10-question batch run lets you compare `gpt-4o-mini` vs `gpt-4o` side-by-side, with persisted cost numbers you can defend at DR #1.
- Your ADR carries a one-paragraph schema-versioning rule that will hold through W6, W13, and W19 as the `Answer` shape evolves.

Time budget: ~2.5 hours of focused work across the two days. The 3-hour live session covers concepts and live demos; this lab is where the code lands on your machine.

Bundle reference

Everything you need is in the W4 bundle. The canonical path on the left is where the file lives in your cohort repo; the bundle filename on the right is what you downloaded.

Canonical path (cohort repo)	Bundle file
------------------------------	-------------

<code>src/pipeline/models.py</code>	<code>AI-RAG_W4_models_starter.py / _reference.py</code>
<code>src/pipeline/cost.py</code>	<code>AI-RAG_W4_cost_starter.py / _reference.py</code>
<code>src/pipeline/pipeline.py</code>	<code>AI-RAG_W4_pipeline_starter.py / _reference.py</code>
<code>src/pipeline/store.py</code>	<code>AI-RAG_W4_store_reference.py</code>
<code>src/api/main.py</code>	<code>AI-RAG_W4_api_main_reference.py</code>
<code>scripts/migrate_store.py</code>	<code>AI-RAG_W4_migrate_store.py</code>
<code>scripts/compare_models.py</code>	<code>AI-RAG_W4_compare_models.py</code>
<code>tests/test_cost.py</code>	<code>AI-RAG_W4_test_cost.py</code>
<code>tests/test_pipeline_w4.py</code>	<code>AI-RAG_W4_test_pipeline_w4.py</code>
<code>data/questions.csv</code>	<code>AI-RAG_W4_questions.csv</code>
<code>docs/adr/0002-api-contract.md</code>	<code>AI-RAG_W4_adr_0002_extended.md</code>
<code>docs/lab4-model-comparison.md</code>	<code>AI-RAG_W4_lab4_comparison_template.md</code>
<code>requirements.txt</code>	<code>AI-RAG_W4_requirements.txt</code>
<i>reference-only</i>	<code>AI-RAG_W4_anthropic_reference.py</code>

Starter vs reference files. Starter files have `TODO` markers and are what you actually edit. Reference files show the finished shape — use them to check your work or to copy a tricky 3-line block. The lab points to a reference file on every sub-step under **Make the change**.

Pre-flight

Before you start: open Day 2 of the slide deck on a second monitor. The Lab Step 1 and Lab Steps 2–4 callouts there mirror this guide. The deck has

the *why*; this guide has the *how*.

Step 0 — Environment (10 min)

0a — Confirm W3 is green before you change anything

What and why

You will modify three files this week — `models.py`, `pipeline.py`, and the SQLite store schema. If anything is broken from W3 you want to know *now*, not three sub-steps in.

Where

Your cohort repo root (the folder with `src/`, `tests/`, `docs/`).

About to change

Nothing — this is a sanity check.

Make the change

Nothing to edit. We're just running the W3 tests and the W3 API.

Run it

```
# Activate your venv from W3
source .venv/bin/activate

# W3 tests must be green before W4 begins
pytest tests/ -q

# Start the W3 API in another terminal (or backgrounded)
uvicorn src.api.main:app --reload --port 8000 &
sleep 2

# Confirm /health responds
curl -s http://localhost:8000/health
echo
```

See

```
.....
[100%]
9 passed in 0.42s
{"status":"ok"}
```

What happened

You confirmed that the W3 state — pipeline, API, tests — is intact. If anything is red, fix W3 *first*. Moving forward with a broken base will mask which week's change introduced a problem.

Stop the uvicorn process (`kill %1` or Ctrl-C in its terminal). You'll start it fresh after each major change in this lab.

0b — Install W4 dependencies

What and why

W4 adds `tiktoken` for the pre-call token-counting that we covered in the deck's Topic 1 mini-topic. Everything else comes from W2/W3.

Where

Repo root.

About to change

`requirements.txt` gains one line — `tiktoken>=0.7,<1.0`.

Make the change

Replace your `requirements.txt` with the W4 version from the bundle (`AI-RAG_W4_requirements.txt`). The only diff vs W3 is the new `tiktoken` entry; you can verify with `diff` if you want to be careful.

Run it

```
pip install -r requirements.txt
python -c "import tiktoken; print(tiktoken.__version__)"
```

See

```
Successfully installed tiktoken-0.7.0
0.7.0
```

(Version may vary; anything in the 0.7–0.9 range is fine.)

What happened

You now have `tiktoken` available. You won't import it heavily this week — it's a 10-min mini-topic — but the dependency is in place for W6 when retrieved context lands and you'll want to guardrail prompt length.

Step 1 — Tool-calling `ask_llm` (60 min)

The big change of Day 1. The W3 `ask_llm` returned a `content` string and a placeholder cost. You're going to make it return a richer structured `Answer` — `content`, `confidence`, and `sources` — by switching to OpenAI's tool-calling API.

Seven sub-steps, all in `src/pipeline/`. The W3 API contract stays put — this is purely an internal upgrade.

1a — Read the W3 baseline

What and why

Before you change anything, you need a clear picture of what's there. The tool-calling switch is small in terms of lines of code but big in terms of *how the function works*. Reading first means fewer surprises.

Where

`src/pipeline/models.py` and `src/pipeline/pipeline.py`.

About to change

Nothing yet.

Make the change

Open both files and skim. Specifically notice:

- `models.py` — the `Answer` model has three fields: `content`, `cost_usd`, `retries`. That's the W3 shape. The cohort goal is to add three optional fields without changing these three.
- `pipeline.py` — find `ask_llm`. It calls `client.chat.completions.create` with just `model` and `messages`. The response is parsed with `resp.choices[0].message.content`. No tools.

If you want to compare against where you'll be by end of Step 1, the final shape is in `AI-RAG_W4_pipeline_reference.py` — but resist the urge to copy-paste yet. Reading the starter first makes the diff land.

Run it

```
# Confirm the W3 ask_llm still works with the fake path
python -c "
import asyncio
from src.pipeline.pipeline import fake_ask_llm
print(asyncio.run(fake_ask_llm('What is RAG?')))
"
```

See

```
[FAKE] What is RAG?
```

What happened

You verified the W3 baseline: `fake_ask_llm` works without any API key. This is your reproducible offline path — every other piece of this lab should keep this fake path working so tests don't depend on the network.

1b — Define the `answer_question` tool schema

What and why

Tool-calling needs a schema that tells the LLM *exactly* what shape to return. The schema is just a Python dict in the OpenAI tool format. When you pass it to `tools=[...]` and force the model with `tool_choice`, the model returns structured arguments instead of free text.

Where

`src/pipeline/pipeline.py`, near the top — there's a `ANSWER_TOOL = None` placeholder in the starter.

About to change

Replace `ANSWER_TOOL = None` with a dict that declares the schema.

Make the change

Open the starter (`AI-RAG_W4_pipeline_starter.py`) and find the TODO at the `ANSWER_TOOL` line. The schema needs three properties:

- `content` — a string (the answer in 2–4 sentences)
- `confidence` — a number between 0 and 1
- `sources` — an array of strings (URLs or source ids; can be empty)

All three should be in the `required` list. The function name is `answer_question`. The wrapping structure is OpenAI's tool format — `{"type": "function", "function": {"name": ..., "parameters": {...}}}`.

The full schema is in `AI-RAG_W4_pipeline_reference.py` (look for `ANSWER_TOOL` near the top). The pattern matches the deck Slide 16 verbatim.

Run it

```
python -c "
from src.pipeline.pipeline import ANSWER_TOOL
print('name      :', ANSWER_TOOL['function']['name'])
print('properties:', list(ANSWER_TOOL['function']['parameters']
['properties'].keys()))
print('required  :', ANSWER_TOOL['function']['parameters']['required'])
"
```

See

```
name      : answer_question
properties: ['content', 'confidence', 'sources']
required  : ['content', 'confidence', 'sources']
```

What happened

You've declared the output shape upfront. The LLM no longer has discretion about how to format the answer — when you wire it in at step 1d, you'll either get back valid `answer_question(content=..., confidence=..., sources=[...])` arguments or you'll get an explicit error. No "Here is your JSON:" preamble to defend against. This is the headline of Topic 2 in the deck.

1c — Extend the Answer Pydantic model

What and why

The schema above declares three return fields. Your `Answer` model needs to *accept* those fields. The key constraint from ADR 0002: the W3 fields must stay unchanged, and any new field must be optional with a default so old clients keep working.

Where

`src/pipeline/models.py`.

About to change

Add three lines inside the `Answer` class.

Make the change

Open the starter (`AI-RAG_W4_models_starter.py`) and read the TODO block inside the `Answer` class. Add three optional fields:

- `confidence: float` — default `1.0`, validated to `[0.0, 1.0]`. Use `Field(default=1.0, ge=0.0, le=1.0)`.
- `sources: list[str]` — default empty list. Use `Field(default_factory=list)` so the default isn't shared across all instances (Pydantic-2 idiom).
- `schema_version: str` — default `"v1"`.

The reference (`AI-RAG_W4_models_reference.py`) shows the exact lines.

Run it

```
python -c "  
from src.pipeline.models import Answer  
# W3 shape – still works (the central guarantee of the ADR)  
a = Answer(content='hi', cost_usd=0.001, retries=2)  
print('W3 shape  :', a.model_dump())  
# W4 shape – uses defaults  
b = Answer(content='hi')  
print('W4 default:', b.model_dump())  
"
```

See

```
W3 shape : {'content': 'hi', 'cost_usd': 0.001, 'retries': 2,
'confidence': 1.0, 'sources': [], 'schema_version': 'v1'}
W4 default: {'content': 'hi', 'cost_usd': 0.0001, 'retries': 0,
'confidence': 1.0, 'sources': [], 'schema_version': 'v1'}
```

What happened

The W3 callable shape — `Answer(content="hi", cost_usd=0.001, retries=2)` — still works without modification. The new fields are present in the output with defaults. This is the canonical example of an *additive* change under the schema-versioning rule you'll formalise in Step 4.

1d — Switch `ask_llm` to use tool-calling

What and why

You have a schema (1b) and a model to fill (1c). Now wire them together. This is the actual moment the LLM stops returning free text and starts returning structured arguments.

Where

`src/pipeline/pipeline.py`, inside the `ask_llm` function — the `# TODO Step 1d` block.

About to change

Replace the `# TODO Step 1d` block with an actual

```
client.chat.completions.create(...)
```

call that passes `tools=[ANSWER_TOOL]` and forces the model to call our tool via `tool_choice`.

Make the change

Open the starter and find the `TODO Step 1d` block. You need to call `client.chat.completions.create` with four arguments:

- `model=settings.model`
- `messages=[{"role": "user", "content": q.question}]`

- `tools=[ANSWER_TOOL]`
- `tool_choice={"type": "function", "function": {"name": "answer_question"}}`

The `tool_choice` parameter is the one that forces the model to call our tool instead of replying in text. Without it the model is free to ignore your tool — which is fine when you're letting it decide whether a tool is needed, but not what you want for a structured-output API.

Assign the result to `resp`. The reference file shows the exact lines.

Run it

Don't run it yet — the parse step (1e) and the cost step (2d) all need to land before this function returns successfully. You'll smoke-test the whole thing in 1f.

See

N/A — this is a code-only sub-step. The visible behaviour is checked in 1f.

What happened

You've made the API call. The response now contains a `tool_calls` list on `message`, not a plain `content` string. Parsing that list is the next sub-step.

1e — Parse the tool call into an Answer

What and why

The LLM's response now contains a structured tool-call object. You need to pull out the JSON arguments, parse them, and build an `Answer` from them.

Where

Same `ask_llm` function in `src/pipeline/pipeline.py`, immediately after the call you just added.

About to change

Replace the `# TODO Step 1e` block with code that:

1. Reads `resp.choices[0].message.tool_calls[0].function.arguments` — this is a JSON string, *not* a dict, because the OpenAI SDK doesn't decode it for you. Pull `import json` at the top of the file.
2. `json.loads` it into a Python dict.
3. Builds an `Answer(content=..., confidence=..., sources=..., retries=attempt, cost_usd=...(see 2d), schema_version="v1")`.

Make the change

Reference: `AI-RAG_W4_pipeline_reference.py`, the `ask_llm` function. Note two things in the reference:

- The defensive check — `if not tool_calls: raise RuntimeError(...)`. This shouldn't fire because we forced `tool_choice`, but if a provider ever misbehaves this gives you a clear error instead of a cryptic `IndexError`.
- The `args.get("sources", [])` instead of `args["sources"]`. The model is *supposed* to always return `sources` because it's required, but a defensive default is cheaper than debugging at midnight.

For `cost_usd`, set it to `0.0` for now — you'll wire real cost in Step 2d.

Run it

```
# Smoke test the parse logic with a hand-rolled JSON payload
python -c "
import json
from src.pipeline.models import Answer
args = json.loads('{\"content\": \"RAG combines retrieval with
generation.\", \"confidence\": 0.9, \"sources\": [\"wiki:RAG\"]}')
ans = Answer(content=args['content'], confidence=args['confidence'],
sources=args['sources'], cost_usd=0.0, retries=0)
print(ans.model_dump())
"
```

See

```
{'content': 'RAG combines retrieval with generation.', 'cost_usd': 0.0,
'retries': 0, 'confidence': 0.9, 'sources': ['wiki:RAG'],
'schema_version': 'v1'}
```

What happened

You've translated the OpenAI-shaped response into your Pydantic `Answer`. The schema discipline pays off here — Pydantic validates `confidence` is in `[0, 1]` and that `sources` is a list of strings. If the LLM ever returns a bad shape you'll get a Pydantic `ValidationError`, not a silent half-broken `Answer` propagating through the pipeline.

1f — Smoke-test via curl

What and why

You've made an end-to-end change. Time to verify it talks to a real OpenAI endpoint and the new fields actually land.

Where

Two terminals — one to start uvicorn, one to curl.

About to change

Nothing in code. You're just running.

Make the change

You need an `OPENAI_API_KEY` exported in your shell. Confirm the `use_fake` flag in your settings is *False* (so the real path runs). Then start uvicorn.

Run it

```
export OPENAI_API_KEY=sk-...      # your key

uvicorn src.api.main:app --reload --port 8000 &
sleep 2

curl -s -X POST http://localhost:8000/ask_batched \
  -H 'Content-Type: application/json' \
  -d '{"question": "What is RAG, in two sentences?"}' \
  | python -m json.tool
```

See

The exact text varies; the *shape* is the point.

```
{
  "content": "Retrieval-Augmented Generation (RAG) combines a
retrieval step over a corpus with a generative LLM, so the model can
ground its output in specific source documents. It reduces hallucination
on factual queries by giving the model relevant context at inference
time.",
  "cost_usd": 0.0,
  "retries": 0,
  "confidence": 0.9,
  "sources": [],
  "schema_version": "v1"
}
```

What happened

The response now carries `confidence`, `sources`, and `schema_version` — fields that simply didn't exist a sub-step ago. `cost_usd` is still 0.0 because you haven't done Step 2d yet. The W3 fields (`content`, `retries`) are still there. A W3 client reading this response would ignore the new fields and keep working. That's the additive-change contract in motion.

1g — Verify `/docs` reflects the new shape

What and why

FastAPI generates an OpenAPI schema from your Pydantic models. The `/docs` page is the easiest way to verify that what you intend to return matches what the API contract says you return.

Where

Your browser, at `http://localhost:8000/docs`.

About to change

Nothing. This is verification.

Make the change

Nothing to edit.

Run it

With uvicorn still running from 1f, open the URL in your browser:

```
http://localhost:8000/docs
```

Click on `POST /ask_batched` → "Schemas" or "Try it out" → expand the response schema for `200`. You should see the six fields.

You can also dump the raw schema from the shell:

```
curl -s http://localhost:8000/openapi.json \
  | python -c "import json,sys; s=json.load(sys.stdin);
print(json.dumps(s['components']['schemas']['Answer'], indent=2))"
```

See

```
{
  "properties": {
    "content": {"type": "string", "title": "Content"},
    "cost_usd": {"type": "number", "title": "Cost Usd", "default":
0.0001},
    "retries": {"type": "integer", "title": "Retries", "default": 0},
    "confidence": {"type": "number", "title": "Confidence", "default":
1.0, "minimum": 0.0, "maximum": 1.0},
    "sources": {"type": "array", "items": {"type": "string"}, "title":
"Sources", "default": []},
    "schema_version": {"type": "string", "title": "Schema Version",
"default": "v1"}
  },
  "required": ["content"],
  "title": "Answer",
  "type": "object"
}
```

What happened

The OpenAPI contract is in sync with the code. Notice how only `content` is `required` — every new field has a default. That's the additive change discipline reflected in the public-facing schema. Step 1 is green.

Stop `uvicorn` (`kill %1`). You'll restart it after Step 2.

Step 2 — Real streaming + real cost + SQLite extension (60 min)

This is the Day 2 hands-on. You'll replace W3's simulated streaming with real OpenAI chunks, replace the `0.0001` cost placeholder with real numbers from `response.usage`, and extend the SQLite store schema to record both. Six sub-steps.

2a — Replace simulated streaming with real LLM chunks

What and why

W3's `/ask` endpoint streamed because we sliced the answer into words and yielded them with `asyncio.sleep(0.05)` between yields. That gave the *feel* of streaming but every token had to wait for the full answer to be generated first. Real streaming yields tokens *as the LLM generates them* — TTFT drops dramatically.

Where

`src/pipeline/pipeline.py`, the `stream_answer` function.

About to change

Replace the `# TODO Step 2a` block — the simulation code — with a real `stream=True` call.

Make the change

Open the starter and find the `# TODO Step 2a` block in `stream_answer`. Make these changes:

- Call `client.chat.completions.create(model=settings.model, messages=[{"role": "user", "content": question}], stream=True)`. Note that this still returns an *awaitable* — you `await` it once to get the stream object, then iterate.
- Iterate with `async for chunk in stream:` — each `chunk` is a small delta from the LLM.
- For each chunk, read `chunk.choices[0].delta.content`. If it's truthy, `yield` it.

Keep the `use_fake=True` branch unchanged at the top of the function — tests rely on the simulated path.

The reference (`AI-RAG_W4_pipeline_reference.py`, the `stream_answer` function) shows the full final shape. Note the defensive

`if not chunk.choices: continue` — sometimes a chunk has empty choices (it's the role-only first chunk), and skipping is cheaper than handling.

Run it

```
uvicorn src.api.main:app --reload --port 8000 &
sleep 2

# --no-buffer is critical – without it curl waits for the full response
curl -s --no-buffer -X POST http://localhost:8000/ask \
  -H 'Content-Type: application/json' \
  -d '{"question": "Explain RAG in 30 words."}'
echo
```

See

You'll see tokens appearing one at a time on your terminal — not in bursts of full words, but a few characters per chunk. The first chunk arrives in ~500 ms; the rest follow at ~10–50 ms intervals.

Retrieval-Augmented Generation (RAG) combines an LLM with a retrieval step over a corpus, so answers are grounded in specific source documents rather than just the model's training data.

What happened

Real streaming is in place. The cohort talked about this in Topic 3 — the *async generator shape* didn't change at all. The function still yields strings. What changed is the source: instead of `await full_call() → split → yield` with `sleep`, it's `await create(stream=True) → async for chunk → yield delta`. Same shape, different filler.

2b — Wire the new `stream_answer` into the `/ask` endpoint

What and why

The W3 `/ask` endpoint already calls `stream_answer` via a `StreamingResponse`. If your `/ask` handler in `src/api/main.py` is unchanged from W3, this sub-step is just verification. If you've drifted from the W3 shape, this sub-step gets you back to canonical.

Where

`src/api/main.py`.

About to change

Replace your file with `AI-RAG_W4_api_main_reference.py` if it has drifted, or read both to confirm they match. The W4 reference is functionally identical to the W3 `/ask` endpoint — same `StreamingResponse(_gen(), media_type="text/plain")` pattern. The only difference is that `_gen()` now reads from a real-streaming source.

Make the change

Confirm your `src/api/main.py` matches the reference. Pay attention to:

- `_gen()` is a nested async function — it has to be, because `StreamingResponse` wants an async iterator.
- `media_type="text/plain"` — *not* `application/json`. A streaming JSON response is its own can of worms; for `/ask` we hand back raw text and let the client assemble it.

Run it

```
# (uvicorn from 2a still running)
curl -s -i --no-buffer -X POST http://localhost:8000/ask \
  -H 'Content-Type: application/json' \
  -d '{"question": "What is a token?"}' \
  | head -20
echo
```

See

```
HTTP/1.1 200 OK
content-type: text/plain; charset=utf-8
transfer-encoding: chunked
...
```

A token is the unit a language model processes – roughly four characters of English text...

What happened

The HTTP status, content-type, and the chunked transfer encoding are all intact from W3. The visible behaviour for downstream consumers is identical. The internals are entirely different — and the W3 contract held without an ADR bump. This is the production lesson from the deck.

2c — Build the cost module

What and why

You need a way to turn `(prompt_tokens, completion_tokens, model)` into a USD number. That's a small, self-contained piece of logic that deserves its own module: `src/pipeline/cost.py`.

Where

New file: `src/pipeline/cost.py`.

About to change

Build out the `RATES` dict and the `compute_cost_usd` function.

Make the change

Start from `AI-RAG_W4_cost_starter.py`. Two TODO blocks:

- Fill in `RATES` with at least these two entries:
 - `"gpt-4o-mini": (0.15, 0.60)` — $0.15/\text{M input}, 0.60/\text{M output}$
 - `"gpt-4o": (2.50, 10.00)` — $2.50/\text{M input}, 10.00/\text{M output}$
- Implement `compute_cost_usd(model, prompt_tokens, completion_tokens):`

- Look up the model in `RATES`. If missing, return `0.0` (don't raise — a missing rate shouldn't break a batch run).
- Compute `(prompt × in_rate + completion × out_rate) / 1_000_000`.
- Return as a float.

The reference (`AI-RAG_W4_cost_reference.py`) shows the final shape, including the `llama3.2:3b` entry for the take-home Ollama activity (free local model).

Run it

```
# Pure unit tests for cost – no network, no API key
pytest tests/test_cost.py -v 2>&1 | tail -15
```

See

```
tests/test_cost.py::TestComputeCostUsd::test_gpt_4o_mini_at_known_token_counts PASSED
tests/test_cost.py::TestComputeCostUsd::test_gpt_4o_at_known_token_counts PASSED
tests/test_cost.py::TestComputeCostUsd::test_zero_tokens_costs_nothing PASSED
tests/test_cost.py::TestComputeCostUsd::test_unknown_model_returns_zero_not_raises PASSED
tests/test_cost.py::TestComputeCostUsd::test_gpt_4o_is_more_expensive_than_gpt_4o_mini PASSED
tests/test_cost.py::TestComputeCostUsd::test_ollama_model_is_free PASSED
tests/test_cost.py::TestRatesTable::test_required_lab_models_present PASSED
tests/test_cost.py::TestRatesTable::test_rates_are_input_output_tuples PASSED
tests/test_cost.py::TestRatesTable::test_output_rate_at_least_input_for_paid_models PASSED
9 passed in 0.05s
```

What happened

You've built a tiny, isolated module that turns usage tuples into USD. This is the kind of small pure function that wants its own file — it has no side effects, no external state, and it's easy to test. The lesson from W3's testing intro: pure functions are gifts. Treat them that way.

2d — Wire real cost into `ask_llm`

What and why

`compute_cost_usd` exists. Now connect it to `ask_llm` so the `Answer` returned by `/ask_batched` carries a real cost number instead of the W3 placeholder `0.0001`.

Where

`src/pipeline/pipeline.py`, inside `ask_llm`, right after you parse the tool call (1e) and before you build the `Answer`.

About to change

Replace the `cost_usd=0.0` you wrote in 1e with a real computation.

Make the change

Find the `# TODO Step 2d` block. Add three lines:

- `usage = resp.usage`
- Pull `usage.prompt_tokens` and `usage.completion_tokens`. Be defensive — if `usage` is `None` for some reason, fall back to `0` for both.
- Call `compute_cost_usd(settings.model, prompt_tokens, completion_tokens)` and pass the result into `Answer(cost_usd=...)`.

Reference: `AI-RAG_W4_pipeline_reference.py`, the `ask_llm` function. The relevant lines are right after the `args = json.loads(args_json)` line.

Run it

```
# Restart uvicorn so it picks up the change
kill %1 2>/dev/null; sleep 1
uvicorn src.api.main:app --reload --port 8000 &
sleep 2

curl -s -X POST http://localhost:8000/ask_batched \
  -H 'Content-Type: application/json' \
  -d '{"question": "What is RAG in two sentences?"}' \
  | python -m json.tool
```

See

The `cost_usd` field is no longer `0.0` — it's a real (small) number.

```
{
  "content": "...",
  "cost_usd": 0.000064,
  "retries": 0,
  "confidence": 0.9,
  "sources": [],
  "schema_version": "v1"
}
```

(Your exact number will differ — it depends on how chatty the model felt on that particular call.)

What happened

That `0.000064` is real. It came from your actual OpenAI bill. If you sum up the `cost_usd` across a thousand calls, you'll get a number close to what shows on the OpenAI dashboard at the end of the day. That's the foundation for the cost budget you'll commit to the ADR in Step 4.

2e — Extend the SQLite store schema

What and why

W2's `answers` table has `id`, `question`, `content`, `retries`, `cost_usd`, `created_at`. W4 wants to record `model`, `confidence`, `sources_json`, and `schema_version` per row — so that next week you can run analytics like "average cost by model" or "median confidence by question type".

The catch: you already have an `answers.db` from W2/W3 with rows in it. You don't want to drop the table and lose them.

Where

`src/pipeline/store.py` (extend it) and run a migration via `scripts/migrate_store.py`.

About to change

- Replace `src/pipeline/store.py` with the W4 reference (`AI-RAG_W4_store_reference.py`). It has the same public API as W3 (`connect`, `save_answer`, `query_results`) plus an `ensure_schema` that runs `ALTER TABLE ... ADD COLUMN` for any missing W4 columns.
- Add a new `scripts/migrate_store.py` (`AI-RAG_W4_migrate_store.py`) that calls `ensure_schema` against your existing DB.

Make the change

Copy both files into place:

```
# (from your repo root)
cp <bundle>/AI-RAG_W4_store_reference.py    src/pipeline/store.py
cp <bundle>/AI-RAG_W4_migrate_store.py     scripts/migrate_store.py
```

The interesting bit in `store.py`: `ensure_schema` reads `PRAGMA table_info(answers)` to see which columns already exist, then runs `ALTER TABLE ... ADD COLUMN` only for the missing ones. Idempotent — safe to rerun.

Run it

```
# Migrate the existing DB
python scripts/migrate_store.py data/answers.db

# Verify the new columns are there
sqlite3 data/answers.db ".schema answers"
```

See

```
Migrated: data/answers.db
  Columns before : ['id', 'question', 'content', 'retries', 'cost_usd',
'created_at']
  Columns after  : ['id', 'question', 'content', 'retries', 'cost_usd',
'created_at', 'model', 'confidence', 'sources_json', 'schema_version']
  Added          : ['model', 'confidence', 'sources_json',
'schema_version']

CREATE TABLE answers (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  question    TEXT NOT NULL,
  ...
  schema_version TEXT DEFAULT 'v1'
);
```

If you rerun `python scripts/migrate_store.py data/answers.db` it should say `Added: (none – already migrated)`. That's the idempotency guarantee — the cohort can run the script anytime without losing data or duplicating columns.

What happened

Your `answers` table now has the W4 shape. Old rows survived — they just have `NULL` in the new columns (except `schema_version`, which got the default `'v1'`). New rows written by the API from this point forward will populate the new columns. This is exactly the production discipline the deck's Topic 5 promised: schema evolves *without* dropping data.

2f — End-to-end smoke test

What and why

You've changed three things — tool-calling, streaming, cost. Run one question through `/ask_batched` and confirm the row in SQLite has all the new columns populated.

Where

Two terminals — `uvicorn` + `sqlite3`.

About to change

Nothing in code. This is verification.

Make the change

Nothing to edit.

Run it

```
kill %1 2>/dev/null; sleep 1
uvicorn src.api.main:app --reload --port 8000 &
sleep 2

curl -s -X POST http://localhost:8000/ask_batched \
  -H 'Content-Type: application/json' \
  -d '{"question": "What is a token?"}' > /dev/null

# Inspect the most recent row
sqlite3 data/answers.db \
  "SELECT model, ROUND(cost_usd, 6), confidence, schema_version,
  substr(content, 1, 40) FROM answers ORDER BY id DESC LIMIT 1;"
```

See

```
gpt-4o-mini|0.000042|0.9|v1|A token is the basic unit a language mode
```

What happened

The new row has every W4 column populated: the model that produced it, the real cost in USD, the confidence the model reported, the schema

version. You can now write SQL queries like "show me the average cost per question for each model" — which is exactly what Step 3 builds on.

Stop `uvicorn` (`kill %1`). Step 3 doesn't need the HTTP layer — it talks to `ask_llm` directly.

Step 3 — Two-model comparison (45 min)

You have a working tool-calling pipeline with real cost tracking. Time to use it. The lesson plan asks you to run the same 10 questions through two OpenAI models and write a short reflection. The script for the heavy lifting is already in the bundle.

3a — Confirm the model is settings-driven

What and why

For the comparison to work, you need to be able to switch models without editing code — just by changing `Settings.model`. Confirm that's the case before you run the comparison script.

Where

`src/pipeline/settings.py` (or wherever your `Settings` class lives) and `src/pipeline/pipeline.py`.

About to change

Probably nothing. `ask_llm` already takes `settings.model` and passes it to the `create` call. This sub-step just verifies that.

Make the change

Skim `ask_llm` in `pipeline.py`. Confirm the `model=` argument to `client.chat.completions.create` is `settings.model`, not a hardcoded string. If you accidentally wrote `model="gpt-4o-mini"` somewhere, change it now.

Run it

```
# Two quick fake-path runs with different model settings – proves the
# wiring works without spending tokens
python -c "
import asyncio
from src.pipeline.pipeline import ask_llm
from src.pipeline.models import Question
from src.pipeline.settings import Settings

for m in ['gpt-4o-mini', 'gpt-4o']:
    settings = Settings(use_fake=True, model=m, openai_api_key='sk-not-
used')
    ans = asyncio.run(ask_llm(Question(question='x'), settings))
    print(f'model={m:<14} ok – content={ans.content[:30]!r}')
"
```

See

```
model=gpt-4o-mini    ok – content='[FAKE] x'
model=gpt-4o         ok – content='[FAKE] x'
```

What happened

You confirmed that switching `Settings.model` doesn't blow anything up.
The next sub-step actually exercises both models against the real API.

3b — Run the comparison script

What and why

The script (`scripts/compare_models.py`) reads the 10 questions from `data/questions.csv`, runs them through `gpt-4o-mini`, then runs them through `gpt-4o`. Each call's result is persisted to SQLite with the correct `model` column. The script prints a summary at the end.

Where

Repo root.

About to change

Nothing — the script is provided as-is in the bundle.

Make the change

Copy the script and the questions CSV from the bundle if you haven't already:

```
cp <bundle>/AI-RAG_W4_compare_models.py scripts/compare_models.py
cp <bundle>/AI-RAG_W4_questions.csv      data/questions.csv
```

Run it

```
# Real API calls – make sure OPENAI_API_KEY is exported
echo "key prefix: ${OPENAI_API_KEY:0:7}..."

python scripts/compare_models.py
```

This runs 20 real API calls (10 questions × 2 models). At `gpt-4o-mini` + `gpt-4o` rates, the total cost should land in the range of 0.01–**0.05** depending on how chatty the models are.

See

The exact numbers vary; the *shape* is what to verify.

```
Running 10 questions through: gpt-4o-mini, gpt-4o
Persisting to: data/answers.db
```

```
→ gpt-4o-mini ...
  done — n=10  cost=$0.000832  avg=$0.000083/q  retries=0  time=18.42s
→ gpt-4o ...
  done — n=10  cost=$0.013648  avg=$0.001365/q  retries=0  time=24.11s
```

```
=====
Model                n      Total $      Avg $/q      Time
-----
gpt-4o-mini          10      0.000832      0.000083     18.42s
gpt-4o                10      0.013648      0.001365     24.11s
```

```
→ gpt-4o cost 16.4× more than gpt-4o-mini on the same questions.
```

What happened

You have two distinct result sets persisted to SQLite. The cost ratio matches the rate ratio almost exactly — about 16×. The latency is similar enough that you wouldn't pick `gpt-4o-mini` for speed alone; you'd pick it for cost.

3c — Eyeball the answers side-by-side

What and why

The numbers tell you about cost and latency. Quality is harder — there's no automated test for "does the answer feel right". You need to read the answers.

Where

Your SQLite db: `data/answers.db`.

About to change

Nothing — read-only inspection.

Make the change

Nothing to edit.

Run it

```
# Side-by-side: same question, both models' content
sqlite3 -header -column data/answers.db <<'SQL'
SELECT
  q.question,
  substr((SELECT content FROM answers WHERE question = q.question AND
model = 'gpt-4o-mini' ORDER BY id DESC LIMIT 1), 1, 60) AS mini_says,
  substr((SELECT content FROM answers WHERE question = q.question AND
model = 'gpt-4o'      ORDER BY id DESC LIMIT 1), 1, 60) AS full_says
FROM (SELECT DISTINCT question FROM answers WHERE model IN ('gpt-4o-
mini', 'gpt-4o')) q
ORDER BY q.question;
SQL
```

See

Ten rows, each with the question and a 60-character snippet from each model's answer. Read them. Look for:

- Questions where `gpt-4o` clearly knows more than `gpt-4o-mini` (e.g. the schema-versioning question often shows the difference).
- Questions where the answers feel identical (most simple definitional questions will).
- Confidence values — do they correlate with what you, the human, find more useful?

What happened

You now have ground truth — your own ground truth — for when the bigger model is worth the cost. This is the kind of evidence you'll show at DR #1 in W5 when a stakeholder asks "why this model not that one".

3d — Write the comparison note

What and why

The lesson plan calls for a 2-paragraph reflection committed to the repo. This document is your evidence for the model choice you'll lock into the ADR in Step 4.

Where

New file: `docs/lab4-model-comparison.md`.

About to change

Create the file from the bundle template.

Make the change

Copy the template and fill in your numbers + observations:

```
cp <bundle>/AI-RAG_W4_lab4_comparison_template.md docs/lab4-model-comparison.md
```

Then edit `docs/lab4-model-comparison.md`:

- Fill the numbers from the `compare_models.py` summary table.
- Write Paragraph 1 — where the gap mattered. Concrete examples beat generalities; quote a short fragment if it helps.
- Write Paragraph 2 — your rough rule for when to reach for `gpt-4o`.

Aim for honesty over conviction. If you couldn't tell a difference for 9 of 10 questions, *that's* the answer — and it's a defensible reason to use `gpt-4o-mini` as the default.

Run it

```
# Sanity check that the file exists and has been edited (not still the template)
wc -l docs/lab4-model-comparison.md
grep -q "<fill in>" docs/lab4-model-comparison.md && echo "STILL HAS TEMPLATE MARKERS — keep writing" || echo "OK — markers replaced"
```

See

What happened

Your model choice is now defensible with evidence. At DR #1 when a stakeholder asks "did you consider the more capable model?", the answer is "yes, here's the comparison, here's why I chose what I chose". That's the production discipline the deck's Topic 4 was pointing at.

Step 4 — ADR update (15 min)

The last 15 minutes. Three tiny edits that make the week's work durable across the rest of the programme.

4a — Add the schema-versioning rule to ADR 0002

What and why

The W3 ADR locks the contract: `/ask`, `/ask_batched`, `/health`, and the W3 `Answer` fields. But the W3 ADR doesn't yet say what happens when the `Answer` shape *needs* to change — which it just did, additively, in Step 1. You're going to write the rule that governs every future shape change in W6, W13, W19, and beyond.

Where

`docs/adr/0002-api-contract.md`.

About to change

Add a new "Schema versioning rule" section to the ADR. The exact wording matters less than the three commitments it expresses.

Make the change

The bundle has the extended ADR pre-written (`AI-RAG_W4_adr_0002_extended.md`) — use it as a template. The substantive content is in three paragraphs:

1. **Every Answer carries a `schema_version` field.** Today's shape is `v1`. Stays `v1` through additive changes.
2. **Breaking changes ship as a new `schema_version`** (`v2`, `v3`, ...). The ADR enumerates what counts as breaking (removed field, renamed field, type change, optional → required, semantic change, error-shape change).
3. **Parallel support.** When `v2` ships, the old version is still served by default; an `X-Schema-Version` header (or `?schema_version=v2`) opts the caller into `v2`. At least two weeks of parallel before `v1` is retired.

```
# If you want to take the bundle ADR wholesale:  
cp <bundle>/AI-RAG_W4_adr_0002_extended.md docs/adr/0002-api-contract.md  
# Then edit to keep your existing W3 ADR text and add only the new  
section.
```

Run it

```
git diff docs/adr/0002-api-contract.md | wc -l
```

See

Some non-zero number of diff lines.

```
84
```

What happened

Your ADR now governs a class of future changes you haven't seen yet. The key sentence — "Additive changes do not require a `schema_version` bump" — is what allowed Step 1 to ship without one.

4b — Add the cost budget and chosen model

What and why

Two soft constraints that DR #1 will probe. They're soft because they're not enforced in code; they're discipline.

Where

Same file: `docs/adr/0002-api-contract.md`.

About to change

Add a short "Cost budget" section and a "Chosen default model" section.

Make the change

The template ADR has both sections pre-filled. Customise:

- **Cost budget.** The default in the template is " ≤ 0.01 per answer on average". Refine this based on Step 3's 'gpt-4o-mini' numbers—your actual cost per answer is probably under 0.001. You might write " ≤ 0.002 / answer, with a hard guardrail at 0.05 / answer single-call".
- **Chosen default model.** The template defaults to `gpt-4o-mini`. Add one sentence justifying it from your comparison note (3d). E.g. "I picked `gpt-4o-mini` because the comparison run showed identical answers on 8 of 10 questions at 16× lower cost."

Run it

```
grep -A2 "Cost budget" docs/adr/0002-api-contract.md
echo "---"
grep -A2 "Chosen default model" docs/adr/0002-api-contract.md
```

See

```
Cost budget (added W4)
Capstone /ask_batched calls cost ≤ $0.002 each on average over the
comparison batch.

---

Chosen default model (added W4)
The default Settings.model is gpt-4o-mini. The Step 3 comparison showed
identical answers on 8 of 10 questions at ~16× lower cost.
```

What happened

Your cost budget is a number you can defend with the Step 3 data, and your model choice is a sentence you can defend with the Step 3 reading. At DR #1 you don't get blindsided by "did you think about cost?" — you already did, in writing.

4c — Verify the W3 contract still holds, then commit

What and why

This whole week was an internal upgrade. The W3 ADR's promise was that the contract — `/ask`, `/ask_batched`, `/health`, the W3 shape of `Answer` — would survive every change. Confirm that. Then commit.

Where

Repo root.

About to change

Nothing — verification + git commit.

Make the change

Nothing to edit. Just run the full test suite and check the OpenAPI schema for the three W3 fields.

Run it

```

# Full test suite – W2, W3, W4 tests all together
pytest tests/ -q

# Re-start uvicorn and verify the W3 fields are still present
kill %1 2>/dev/null; sleep 1
uvicorn src.api.main:app --reload --port 8000 &
sleep 2

curl -s http://localhost:8000/openapi.json \
  | python -c "
import json, sys
schema = json.load(sys.stdin)['components']['schemas']['Answer']
['properties']
for w3_field in ('content', 'cost_usd', 'retries'):
    assert w3_field in schema, f'{w3_field} missing – W3 contract
BROKEN'
    print(f'  {w3_field:<12} present ✓')
"

kill %1; sleep 1

git add docs/adr/0002-api-contract.md docs/lab4-model-comparison.md
git add src/pipeline/models.py src/pipeline/pipeline.py
src/pipeline/cost.py
git add src/pipeline/store.py src/api/main.py scripts/migrate_store.py
git add scripts/compare_models.py tests/test_cost.py
tests/test_pipeline_w4.py
git status

```

See

```
.....
```

```
[100%]
```

```
25 passed in 1.12s
```

```
content      present ✓
```

```
cost_usd     present ✓
```

```
retries      present ✓
```

```
On branch main
```

```
Changes to be committed:
```

```
modified:   docs/adr/0002-api-contract.md
```

```
new file:   docs/lab4-model-comparison.md
```

```
modified:   src/pipeline/models.py
```

```
modified:   src/pipeline/pipeline.py
```

```
new file:   src/pipeline/cost.py
```

```
...
```

What happened

- All tests pass (W2 + W3 + W4 — additive changes don't break older tests).
- The three W3 fields are present in the OpenAPI schema — the contract held.
- Your repo is staged for a clean commit:

```
git commit -m "W4: tool-calling, real streaming, real cost, schema versioning"
```

You're done. The capstone now has a defensible cost story, a clean contract, and an internal architecture that's ready for W5's evaluation work and DR #1.

After Day 2

Optional take-home (~30–45 min). The deck called out Ollama as the local-model path. The activity document `AI-RAG_W4_Activity.md` walks through installing Ollama, pulling `llama3.2:3b`, integrating it as a third option in your pipeline, and running the same 10 questions through it. The local cost is `$0`. The quality gap vs `gpt-4o-mini` is the point of the exercise.

Pre-read for W5. Skim the OpenAI Evals quickstart. We use a smaller custom harness in W5, but seeing what the heavyweight version looks like will make the "why we built it ourselves" framing land.

Design Review #1 is in W5. Have your `docs/lab4-model-comparison.md` and the `Cost budget` paragraph in your ADR ready. A stakeholder may ask "why this model, what's the cost, what's the contract" — your week 4 work is the answer to all three.